Care Access Navigator

Incident Response 1 — Task 1: Incident Response Environment Setup

SOC / SIEM lab, detection rules, packet capture and memory-forensics tooling

Document ID	CAN-IR1-T1-ENV-001
Author	Akhila Chennamaneni
Date	October 15, 2026
Environment	Apple M4 (arm64), macOS; Docker Desktop 29.x; no administrator rights
Lab host	parrot-sim — Debian 12 (bookworm), stand-in for Parrot OS Security
Data classification	Synthetic lab data only — no real ePHI, no live system attacked
Status	Delivered (equivalent lab; see Scope & Substitution)

1. Objective

This task builds and documents the incident-response lab that the rest of the Incident Response course runs on: a monitored host, a SIEM-style detection layer with custom rules, network packet capture, and memory-forensics tooling. The deliverables are (1) a configured lab host with centralized log collection, (2) three custom detection rules in Wazuh rule syntax with evidence that each one fires, (3) Wireshark capture filters, and (4) a configured Volatility 3 install for later memory analysis.

2. Scope and tool substitution

The Incident Response 1 rubric names **Wazuh SIEM on Parrot OS inside VirtualBox**, with a Wazuh dashboard and macOS log ingestion. This lab delivers the same capabilities with an equivalent, honestly-labelled setup, because of two hard constraints on the build machine:

- **No administrator rights.** VirtualBox and the macOS Wazuh agent both require admin installation, which this account does not have. Docker Desktop is available and needs no admin rights per run.
- **Apple-silicon (arm64) host.** There is no maintained Parrot OS VirtualBox image for arm64. Parrot is built on Debian, so a Debian 12 container (`parrot-sim`) is used as the stand-in OS.

This is the “equivalent lightweight SIEM/log-analysis simulation” the capstone design document allows for, and it is recorded as an approved scope decision in `docs/adr/ADR-SIEM-choice.md` and the project exception register. What is **real** in this lab: the host OS and services, the sshd/sudo log format, the Wazuh rule syntax, the packet capture, and the Volatility 3 install. What is **simulated**: the SIEM is a Python rule engine (not a Wazuh manager + dashboard), and the suspicious activity is injected as sample log lines, which is exactly how Wazuh's own `wazuh-logtest` validates rules.

> **Action required:** Before final grading, confirm in writing that the instructor accepts this substitution for the Wazuh-, Parrot- and VirtualBox-specific rubric items. The identity and detection evidence stands on its own; only the named platform differs.

3. How this meets the Environment Setup rubric

Rubric item	Delivered as	Evidence
SIEM installed/configured	Wazuh-syntax rules evaluated by detect_engine.py (SIEM-equivalent)	§6, §7
Basic agent / host deployment	parrot-sim host on an isolated Docker network	§4
Log collection from the host	rsyslog → /var/log/auth.log (sshd, sudo)	§5
3 custom alert rules	Rules 100101, 100102, 100110 (brute force, compromise, priv-esc)	§6
Wireshark with capture filters	5 saved tshark/Wireshark capture filters + live 63-packet capture	§8
Volatility configured	Volatility 3 Framework 2.7.0, 25 Linux plugins, config file	§9
Parrot + macOS logs	Parrot-sim auth.log (real) + macOS install.log sample (real)	§10

4. Lab architecture and isolated network

The lab runs two logical zones on a Docker **--internal** network (no route to the internet — the equivalent of a VirtualBox “Internal Network” for segmentation and containment). The monitored host plays the victim clinical workstation; a separate source address (10.66.0.66) represents the attacker in the scenario the rules detect.

Figure 1. Lab topology

```
Docker --internal network  ir1-labnet  (10.66.0.0/24, no internet route)
+-------------------------------------------------------------------+
|                                                                   |
|   parrot-sim  10.66.0.10   (Debian 12 = Parrot OS stand-in)     |
|     - sshd (verbose auth logging)   - rsyslog -> auth.log       |
|     - tshark / Wireshark filters    - Volatility 3              |
|     - detect_engine.py  (loads local_rules.xml)                 |
|                                                                   |
|   attacker source 10.66.0.66  (scenario origin in sample log)   |
+-------------------------------------------------------------------+
      host: Apple M4 macOS + Docker Desktop (also a log source)
```

1 Build the host and the isolated network

Real output from `lab/run_lab.sh` (Docker server version, the isolated network, and the two hosts on it):

transcripts/01_build_and_network.txt

```
$ docker version --format '{{.Server.Version}} / {{.Server.Arch}}'
29.7.2 / arm64

$ docker build -t ir1-parrot-sim:1.0 -f Dockerfile.parrot-sim .









# isolated lab network (--internal = no internet route; VirtualBox 'Internal Network' equivalent)
$ docker network create --internal --subnet 10.66.0.0/24 ir1-labnet
c6eb5cccd0a361e0f6359aca62519393bd9f95d151eb09753c1bb4ebd12e2b45
$ docker run -d --name parrot-sim --hostname parrot-sim --network ir1-labnet --ip 10.66.0.10
    ir1-parrot-sim:1.0
```

5. Log collection

rsyslog routes authentication events to `/var/log/auth.log`, the file a Wazuh agent would tail as a `localfile` source. sshd runs with `LogLevel VERBOSE` so both failed and accepted passwords are recorded. Two lab accounts are created: `nurse_jsmith` (a clinical, non-privileged role) and `it_admin` (in the sudo group).

transcripts/02_host_and_logging.txt

```
$ service rsyslog start ; pgrep -a rsyslogd
20 rsyslogd

$ grep -E 'auth|authpriv' /etc/rsyslog.conf  (auth.log routing = Wazuh 'localfile' source)
55:*.*;auth,authpriv.none       -/var/log/syslog
60:auth,authpriv.*          /var/log/auth.log

# create the lab accounts used in the scenario
$ useradd nurse_jsmith (clinical, non-sudo) ; useradd it_admin (sudo)
uid=1000(nurse_jsmith) gid=1000(nurse_jsmith) groups=1000(nurse_jsmith)
uid=1001(it_admin) gid=1001(it_admin) groups=1001(it_admin),27(sudo)

# start sshd with verbose auth logging (so failed/accepted passwords are recorded)
$ ssh-keygen -A ; sshd ; ss -ltn | grep :22
LISTEN 0      128          0.0.0.0:22         0.0.0.0:*
LISTEN 0      128             [::]:22            [::]:*
```

6. Three custom detection rules

The rules are written in Wazuh local-rules syntax and live in `wazuh-sim/local_rules.xml`. A real Wazuh manager would load this file unchanged. They build on a base SSH-failure rule (100100) and encode the scenario of an off-hours attack on a clinical account:

Rule	Level / severity	Fires when	MITRE ATT&CK
100101 — SSH brute force	10 / High	≥6 failed logins from one source IP within 120 s	T1110
100102 — login after brute force	12 / Critical	a successful login from a source that just tripped 100101	T1110, T1078

Rule	Level / severity	Fires when	MITRE ATT&CK
100110 — privilege escalation	12 / Critical	a clinical (nurse_*) account invokes sudo to root	T1548.003

wazuh-sim/local_rules.xml (Rule 100101, the brute-force rule)

```
<!-- CUSTOM RULE 1 - SSH brute force.
       Fires when rule 100100 matches >=6 times from the same source IP within 120 s. -->
  <rule id="100101" level="10" frequency="6" timeframe="120">
    <if_matched_sid>100100</if_matched_sid>
    <same_source_ip />
    <description>SSH brute-force attack: >=6 failed logins from $(srcip) in 120s</description>
    <mitre>
      <id>T1110</id>
    </mitre>
    <group>authentication_failures,attack,hipaa_164.308.a.5.ii.c,</group>
  </rule>
```

7. Rule validation (sample-log injection)

Each rule is validated by replaying a synthetic `auth.log` through the rule engine — the same approach as Wazuh's `wazuh-logtest`. The sample contains eight failed logins against `nurse_jsmith` from 10.66.0.66, then a successful login, then a sudo-to-root attempt. A benign admin login is included as a control that must *not* alert.

transcripts/06_detection_rules.txt (rule file check + engine run)

```
$ xmllint --noout local_rules.xml  (well-formed check)
local_rules.xml: well-formed XML

$ grep '<rule id' local_rules.xml
<rule id="100100" level="5">
<rule id="100101" level="10" frequency="6" timeframe="120">
<rule id="100102" level="12" frequency="1" timeframe="300">
<rule id="100110" level="12">

$ python3 detect_engine.py --rules local_rules.xml --log sample_auth.log --out ../evidence
```

The alerts the engine raised (full set in `evidence/alerts.csv`; the six Low base-rule hits are omitted here):

Time (UTC)	Rule	Severity	Source	Detection
02:11:14	100101	**High**	10.66.0.66	SSH brute-force attack: >=6 failed logins from 10.66.0.66 in 120s
02:11:24	100102	**Critical**	10.66.0.66	Successful SSH login from 10.66.0.66 right after a brute-force burst - possible account compromise
02:12:48	100110	**Critical**	local	Privilege-escalation attempt: clinical account used sudo to reach root

> **Result:** All three custom rules fired on the correct events, in order, and the benign `it_admin` login produced no alert — so the rules detect the attack without a false positive on normal activity.

8. Wireshark capture filters and live capture

Five capture filters are saved to the Wireshark profile (`~/.config/wireshark/cfilters`), usable from both the Wireshark GUI and `tshark`. Each filter is compile-checked, then a live capture on port 22 records real SSH traffic.

transcripts/03_wireshark.txt (version, interfaces, saved filters, compile checks)

```
$ tshark --version
TShark (Wireshark) 4.0.17 (Git v4.0.17 packaged as 4.0.17-0+deb12u3).

$ tshark -D   (capture interfaces)
1. ciscodump (Cisco remote capture)
2. dpauxmon (DisplayPort AUX channel monitor capture)
3. randpkt (Random packet generator)
4. sdjournal (systemd Journal Export)
5. sshdump (SSH remote capture)
6. udpdump (UDP Listener remote capture)
7. wifidump (Wi-Fi remote capture)

# install the saved Wireshark capture filters (used by both Wireshark GUI and tshark)
$ cp cfilters ~/.config/wireshark/cfilters
"IR - SSH only" tcp port 22
"IR - attacker host" host 10.66.0.66
"IR - TCP SYN (scan / new sessions)" tcp[tcpflags] & tcp-syn != 0
"IR - exclude lab mgmt" not port 2375 and not port 2376
"IR - DNS" udp port 53
```

transcripts/04_capture.txt (live capture of SSH traffic)

```
# capture SSH traffic to prove the filter works end to end. tshark runs with capture
# filter 'tcp port 22'; we then generate one ordinary SSH handshake to parrot-sim.
$ tshark -i any -f 'tcp port 22' -a duration:12 -w /evidence/pcap/ssh_capture.pcapng &
$ ssh nurse_jsmith@10.66.0.10  x3  (three handshakes over eth0 -> real port-22 packets)
Warning: Permanently added '10.66.0.10' (ED25519) to the list of known hosts.
nurse_jsmith@10.66.0.10: Permission denied (publickey,password).
nurse_jsmith@10.66.0.10: Permission denied (publickey,password).
$ capinfos / tshark -r ssh_capture.pcapng  (packet count + first/last)

==================================
|                                |
| Duration: 2.370 secs           |
|                                |
packets captured: 63
```

Result: 63 packets were captured to `evidence/pcap/ssh_capture.pcapng` with the `tcp port 22` filter, confirming the capture pipeline works end to end.

9. Volatility 3 (memory forensics)

Volatility 3 is installed and configured with symbol and cache directories for the memory analysis performed in Task 4. 25 Linux plugins are available, including the ones used to investigate a compromised host (`linux.pslist`, `linux.pstree`, `linux.bash`, `linux.sockstat`).

transcripts/05_volatility.txt

```
$ vol --help | head -1   (framework version)
Volatility 3 Framework 2.7.0

# configure symbol/cache dirs and record the Linux memory-forensics plugins available
$ vol -h | grep linux.   (memory plugins ready for Task 4)
linux.bash
linux.capabilities
linux.check_afinfo
linux.check_creds
linux.check_idt
linux.check_modules
linux.check_syscall
linux.elfs
linux.envars
linux.iomem
linux.keyboard_notifiers
linux.kmsg
$ wc -l linux_plugins.txt
```

10. Parrot and macOS log sources

The rubric asks for both Parrot and macOS logs. The Parrot-sim side is the real `auth.log` used throughout this task. For the macOS host, the unified log `auth` store needs administrator rights to read; the readable, real macOS log used here is `/var/log/install.log`. A full macOS `log stream` feed is noted as a follow-up for when admin rights are available.

evidence/macos/macos_install_log_sample.log (real macOS log, first lines)

```
# macOS host log sample - Care Access Navigator IR1 Task1
# host: DAEDMAC1009.local  os: macOS 26.3.1 (arm64)
# source: /var/log/install.log (readable without admin; the unified auth log store needs sudo)
# ---
2026-10-15 11:47:57-04 DAEDMAC1009 softwareupdated[91894]: Adding client SUUpdateServiceClient
    pid=93360, uid=0, installAuth=NO rights=(), transactions=0 (/usr/libexec/mdmclient)
2026-10-15 11:47:57-04 DAEDMAC1009 softwareupdated[91894]: SoftwareUpdate: Fired early. Next
    check=10/16/26, 12:00AM (interval=86280.000000, A/C=NO)
2026-10-15 11:48:39-04 DAEDMAC1009 softwareupdated[91894]: Removing client SUUpdateServiceClient
    pid=93360, uid=0, installAuth=NO rights=(), transactions=0 (/usr/libexec/mdmclient)
2026-10-15 12:23:42-04 DAEDMAC1009 softwareupdated[91894]: Removing client SUUpdateServiceClient
    pid=87425, uid=502, installAuth=NO rights=(), transactions=0 (/System/Library/PrivateFrameworks/Soft
    wareUpdate.framework/Versions/A/Resources/SoftwareUpdateNotificationManager.app/Contents/MacOS/Softw
    areUpdateNotificationManager)
2026-10-15 12:23:42-04 DAEDMAC1009 softwareupdated[91894]: Removing client SUUpdateServiceClient
    pid=87425, uid=502, installAuth=NO rights=(), transactions=0 (/System/Library/PrivateFrameworks/Soft
    wareUpdate.framework/Versions/A/Resources/SoftwareUpdateNotificationManager.app/Contents/MacOS/Softw
    areUpdateNotificationManager)
```

11. Evidence integrity and chain of custody

Every artifact is hashed with SHA-256 at the end of the run into `evidence/EVIDENCE-INDEX-SHA256.txt`. This is the baseline that Task 4 (Digital Evidence) extends with a formal chain-of-custody record.

SHA-256 (first 16 hex)	Artifact
ea38abbfcd03c7c2…	evidence/alerts.csv
d049710257866fe6…	evidence/alerts.json

SHA-256 (first 16 hex)	Artifact
bb968dac8bd25d7d…	evidence/logs/auth.log
babdd39ddb0607c2…	evidence/macos/macos_host_info.txt
25f649eb4948e306…	evidence/macos/macos_install_log_sample.log
748583eb9f86783e…	evidence/pcap/capture.log
43995315ec5e26d5…	evidence/pcap/ssh_capture.pcapng
8bf9a48676565164…	evidence/transcripts/01_build_and_network.txt
83e52096da429dde…	evidence/transcripts/02_host_and_logging.txt
839030ca4fb54964…	evidence/transcripts/03_wireshark.txt
bca4dc3d941f9a71…	evidence/transcripts/04_capture.txt
3ffe04fcd2dd768a…	evidence/transcripts/05_volatility.txt
033e26d16d4de99c…	evidence/transcripts/06_detection_rules.txt
bbef4f85478ea53b…	evidence/volatility/linux_plugins.txt
86345e8ba44a658b…	evidence/volatility/vol_config.ini
6761c8a1f0cd44a2…	evidence/wireshark/cfilters

12. How to reproduce

The whole lab is one script and needs only Docker Desktop:

Reproduce

```
cd "incidence response_1/task1_environment_setup/lab"
bash run_lab.sh          # builds host, configures tools, runs rules, writes evidence/

# tear down when finished:
docker rm -f parrot-sim && docker network rm ir1-labnet
```

Files: `lab/` (Dockerfile + run_lab.sh), `wazuh-sim/` (local_rules.xml, detect_engine.py, sample_auth.log, cfilters), `evidence/` (transcripts, pcap, alerts, hashes).